A screenshot or a sheet without your student ID or your own address in it is worth zero marks. It could have come from anyone.

Homework 2

Blockchain Basics and Platforms • Inha University in Tashkent • Fall 2026

Assigned	25.10.2026	**Instructor**	Yusupov Jalolliddin
Due	09.11.2026, 23:59	**Weight**	100 points = 5% of the course grade
Submit	One ZIP on eClass	**Work**	Individual

What you submit

One ZIP file, containing exactly this:

```
SOC4190_HW2_<StudentID>_<Surname>.zip
├── report.pdf   ← the answers, the screenshots, the photos of your handwriting
└── src/
    └── txid.py  ← Part 2, exercise 2.4, typed in by you
```

The report is **one PDF**. Write it in Word or LibreOffice and export to PDF. Number your answers exactly as this handout numbers the questions — 1.1, 2.3, 3.4 — so we can find them. If you used an AI tool, say so in one line at the top of the report: which tool, for what.

> **If a website is down.** If mempool.space or Etherscan does not load, use another explorer (blockstream.info for Bitcoin, any Sepolia explorer for Sepolia) and write in the report which one you used. A site that is down is not an excuse for an empty answer.

Your own numbers

As in Homework 1, almost every task uses numbers taken from **your student ID**. Work them out once, write them at the top of your report, and use them everywhere.

Name	How to get it	Example: ID U2310123
Block height **H2**	840,000 plus the last five digits of your ID, read as a number	10123 → 850,123
Data string **D**	your surname in CAPITAL letters, exactly as on your eClass account	YUSUPOV
Transaction **T**	the **second** transaction listed in block H2 on mempool.space (the first one is the coinbase transaction — Lecture 7; we meet it in 3.4)	(you look it up)
Variant **A / B / C**	from the last digit of your ID: 0–3 → A, 4–6 → B, 7–9 → C	If last digit of your ID is you will use A variants below: **3** → **A**
Account 1 address	the address of Account 1 in your MetaMask	(your own)

Every block with a height of 840,000 or more was found after the April 2024 halving, so every H2 is a real block from 2024–2026. Your block H2 is yours: the explorer address looks like `https://mempool.space/block/<H2>` (you can also type the number into the search box).

The course address (the address from the faucet lecture, registered to the course) is:

0x02B75C782186B7615730A246D0b366d54Fa38772

The rules for screenshots

A screenshot only counts if **all three** of these are true:

1. It shows your own work: your Account 1 address, your block H2, your D, or — in a terminal — your student ID. Before running txid.py, type echo U2310123 (your own ID) in the same terminal window so it is in the screenshot. For the mempool.space front page (2.6), the computer's clock must be visible. On an explorer the browser address bar must be visible. On the Human Miner page, your D must be visible in the Data box.
2. The **whole result** is visible, not a cropped corner.
3. It is **readable**. Use the screenshot tool of your computer, not a phone photograph of the monitor.

The rules for handwriting

The exercises marked ✍ are done **on paper, by hand**. Write your name and student ID at the top of every sheet, photograph or scan the sheets, and put the pictures in the report. Typed tables and typed calculations score zero for these exercises. Show the arithmetic, not only the result: we mark the steps. Exercises marked 💻 are done on the computer.

Part 1 — Your first transaction on Sepolia [30 points]

From Lecture 5. Sepolia is the test network (testnet): the coins are free and worth nothing, the mechanics are real. One warning for the whole Part: **never send a private key or a seed phrase to anybody, and never put one in your report.** If you have ever pasted your twelve words anywhere, make a new wallet.

1.1 — The first transaction, finally [8]

Open MetaMask, choose the Sepolia network, and get free test ETH from a faucet (the website that gives out test coins). The lecture showed Google's Sepolia faucet; any Sepolia faucet is fine. Then:

1. Send **0.001 Sepolia ETH** from Account 1 to the course address (above).
2. Create **Account 2** in the same MetaMask (the “Add account” button) and send **0.002 Sepolia ETH** from Account 1 to Account 2.

Each transaction gets a page on `sepolia.etherscan.io`. Open both pages. They show many fields. **Ignore every other field on that page; we have not learned them yet.** Fill in exactly these five:

	Transaction 1 (to the course)	Transaction 2 (to Account 2)
From		
To		
Value		
Block (number)		
Status		

Hand in: for each of the two transactions, a screenshot of its page on sepolia.etherscan.io **with the address bar visible**; the filled table above; and one screenshot of MetaMask showing the sender address, which must match the From field.

1.2 — Two accounts, one seed [5]

Do not show your Secret Recovery Phrase and never show a private key. A screenshot that shows either one scores zero, and you must make a new wallet.

Hand in: one screenshot of MetaMask with Account 1 and Account 2 both visible (the addresses, nothing secret); and two sentences: why does one Secret Recovery Phrase give many addresses, and what exactly is lost if the phrase is lost?

1.3 — Restore test [5]

Install MetaMask in a **second browser or a new browser profile**, choose “Import an existing wallet”, and enter your phrase. Check that the *same* Account 1 address appears. Then remove that new profile — only the new one, never your main browser or your original MetaMask.

> **Safety.** Do this only on your own computer, never on a shared one. The test-network phrase holds no real money — but treat it as if it did, and do not paste the phrase into any website.

Hand in: a screenshot of the restored wallet that shows the same Account 1 address (no phrase in the picture); and one sentence: where does the account actually live — in the browser, in MetaMask's company, or on the chain?

1.4 — Break your own address [6]

Take the course address. Change **one character in the middle** of it (keep all other characters, with their capital and small letters, exactly as they were). Then:

1. paste the broken address into MetaMask's “Send to” field;
2. open `https://sepolia.etherscan.io/address/<your broken address>` in the browser.

Write down what each of the two did: refused it, warned you, or quietly showed an address page with no history. Do **not** send anything to the broken address.

Hand in: two screenshots, one for each reaction; and one sentence: why would the same typo have slipped through if the address had been written in all small letters?

1.5 — Private key = 1, today [6]

Lecture 5's war story: if the private key is the number 1, anybody can compute the address. Both addresses below were computed from the private key 1 by the teaching team on 25.10.2026 (the Bitcoin address from the compressed public key):

```
Bitcoin   1BgGZ9tcN4rm9KBzDn7KprQz87SZ26SAMH
Ethereum  0x7E5F4552091A69125d5DfCb7b8C2659029395Bdf
```

Open the Bitcoin address on `mempool.space/address/…` and the Ethereum address on `etherscan.io/address/…` (the main network, not Sepolia). **Look, do not send.**

	Balance now	Number of transactions	Date of the latest one
Bitcoin address			
Ethereum address			

Hand in: the filled table with the date and time you looked, and two sentences: why does money sent to such an address disappear within seconds, and who takes it?

Part 2 — Reading Bitcoin transactions [40 points]

From Lecture 6. Open your transaction **T** on mempool.space (block H2, second transaction in the list). Write its full txid at the top of your report. Dark humour for the road: a transaction is a very polite thing — it lists exactly where every satoshi came from and exactly where it goes, and it never forgets.

Worked example for student ID U2310123. I checked these on mempool.space today.

Step	Value
Last five digits of the ID	10123
H2 = 840,000 + 10123	850,123
Block page	https://mempool.space/block/850123
First transaction in the list	the coinbase, paid to AntPool
Second transaction = T	`405627cf5958a99244a54107014b3f92de2c3b7cc35c29678bee3ec55daa34cd`

> **If T is big.** Some transactions have dozens of inputs or outputs. If T has more than 8 inputs or more than 8 outputs, add up the first eight by hand, then take the rest from the explorer's total and say so on your sheet. You still show the arithmetic for the eight.

2.1 ✍ — Fee = inputs − outputs [7]

Hand in: a screenshot of the transaction page, with the txid visible; and, by hand: every input amount and every output amount in **satoshi** (1 BTC = 100,000,000 satoshi; the explorer shows BTC, so convert), the sum of the inputs, the sum of the outputs, and their difference. Then the fee the explorer shows, for comparison, and one sentence: who receives the difference?

2.2 — Spent or unspent [4]

On the page of T, the explorer marks each output: one that someone has already used as an input of a later transaction is **spent**, with a link to the transaction that spent it; one that nobody has used yet is marked **unspent**. Look for that mark on the right side of each output.

Hand in: a screenshot showing the outputs with their markers; the list of which outputs of T are still in the UTXO set at the moment of the screenshot (write the date and time); and two sentences: what is the UTXO set, and how does it catch a double spend?

2.3 ✍ — Decode a raw transaction by hand [10]

Below are three real **legacy** transactions (legacy = the old format from before SegWit, still allowed today: no witness, nothing hidden after the version) from the Bitcoin main network, blocks 870,000 to 940,000 (2024–2026). Take the one for your **variant** (A, B or C — see “Your own numbers”). Each has exactly one input. The text is the **raw transaction** in hexadecimal: two hex characters are one byte, and the lines below are cut every 64 characters only so it fits on the page — the cuts mean nothing.

Variant A — 1 output, 191 bytes, block 940,000

```
txid: 4bfd9c9531ad7d0f72d269b2b3e0aba61bf0984aa9d388601a031e9abac9b5d7

010000000130f1fe31ac630e350801f9c9af274ff5092a72c9b5729d3ac8e1f6
293467a361000000006a47304402203435a17ee20daef05385ac9afa86a26d96
04e7a73bc8fb02873922787d9aa2d602203ff00137b8d72f6ea35039a0395699
80137a63d0a10fe1beb6d5bd64b3d6a39401210277dad6a3a0eeea22fce6a9bc
eb851359b197b14be0b75cce41ef91edb296efb0ffffffff01d2950c00000000
001976a914bf5a83674e05f6ce68ce8c09c9b6069006c97ad488ac00000000
```

Variant B — 1 output, 191 bytes, block 870,000

```
txid: d771ab479ba6e000654e504451222e145526f7e22520e35543b92df5a4f52b81

0200000001bdbb43205616692050eceaabbd29c34a4163113cc6fb39bac99b7e
23069b617c430000006a47304402204ce64f046f3d8135de412475084e594d82
98b187f29d5398af5f1c5f84d6539f02205778a7c7e781a26dcfb2880abbf7d5
29bf3d2bca0feba6270d9ce4dd05de9cef0121024a9129b79186615674703f7d
6ca78e92f992ef6d59aae07255002cd36b9f8b23ffffffff016dd60e00000000
001976a914e13d951a695b19b98bcd9d060ae5d18b3f9bb9cf88ac00000000
```

Variant C — 2 outputs, 226 bytes, block 940,000

```
txid: a2103b5ca765276db3105467222ffc038c6c35d0fd84ed1a6175cf5304c74597

01000000014995a361d0e39e52680ebc2056c5598f3b61497699197b4dd2ff52
5648cfb77a010000006b483045022100dbc6734126f0c48f8d0cd65906d685e2
67e2cef6c0702a3c0ae9b25fbf167e3802207d0107d2673f2b5a9aad67eef5b4
2c4e03eda211a46f45d0b03a5e0f1f9da3da012103d804257ec9d37db087454b
d70733e39fe65008d62ee8d25036f1cf4f7e3b52fdffffffff029d8601000000
00001976a91480078dd54f27ab2c3749a7cb1089028554cb017288acea330100
000000001976a914b5edee28418f066a9585c9f303185f822699fd4088ac0000
0000
```

You can fetch the same hex yourself: `https://mempool.space/api/tx/<txid>/hex`. With a pencil, **bracket every field in the hex on your own printout and label it**, in this order (this is the map from Lecture 6):

Field	Bytes	Meaning
Version	4	which rule set; little-endian (the smallest byte comes first)
Input count	1 (usually)	how many inputs follow
Previous txid	32	the transaction whose output is spent — written **reversed** (bytes in the opposite order to the txid you see on the explorer)
Output index	4	which output of that transaction; little-endian
Script length	1 (usually)	how many bytes the unlocking script has
Unlocking script	as given	signature and public key; just mark where it begins and ends

Sequence	4	a 4-byte field at the end of each input; copy it, do not decode it
Output count	1 (usually)	how many outputs follow
Value	8	the amount in satoshi; little-endian
Script length	1 (usually)	how many bytes the locking script has
Locking script	as given	the conditions to spend it (here: P2PKH, 25 bytes)
Locktime	4	the last 4 bytes of the transaction

Then turn each **value** from little-endian into a number: reverse its bytes, read it as hexadecimal, and convert to decimal satoshi. Compare with the amounts shown on the explorer page of your variant's txid.

Hand in: the photographed sheet with all fields bracketed and labelled; the decoded value (or both values, for variant C) in satoshi, written next to the amount the explorer shows; and one sentence saying how you recognised the previous txid as reversed.

2.4 💻 — txid.py [10]

Lecture 6: the transaction ID is the SHA-256 of the raw transaction, applied twice, with the bytes of the result reversed. Fetch the raw hex of your variant (the API address in 2.3), save it into a text file, for example `variant.hex`, and **type in** this program.
Do not copy and paste it, and read each line as you type:

```
import hashlib, sys
raw = bytes.fromhex(open(sys.argv[1]).read().strip())
once = hashlib.sha256(raw).digest()
twice = hashlib.sha256(once).digest()
print(len(raw), "bytes  txid:", twice[::-1].hex())
```

Run it as `python3 txid.py variant.hex`. What we check:

	Requirement	Points
a	the program prints the byte count and the txid	3
b	the txid printed equals the explorer's, character by character (variant: A 191, B 191 or C 226 bytes)	3
c	you change one hex character in a copy of the file to another hex digit (0–9 or a–f), run again, and screenshot the new txid next to the old one	2
d	two sentences: what is transaction malleability, and why does the signature not protect the txid?	2

Hand in: screenshots of (a) and (b) with the terminal prompt and the explorer page both visible, the screenshot of (c), the sentences for (d), and the file `txid.py` in the folder `src/` of your ZIP.

> **Optional, 0 points — the SegWit surprise.** Your variant is legacy, so the simple program works on it. Now fetch the hex of your own **T** the same way and run the program on it. Does the txid match the explorer's? What did you get, and why? (Hint: the API's hex of a SegWit transaction contains the witness, and the txid is computed without it.) Write two lines if you tried; nobody loses a point for skipping it.

2.5 ✍ — Measure the SegWit discount [6]

In your block H2 find **two transactions**, each with 1–2 inputs and 1–2 outputs: one whose inputs are **legacy** (addresses starting with 1 or 3, no witness) and one whose inputs are **SegWit** (addresses starting with bc1q). Scroll the transaction list of the block, or use the page of the block and open transactions one by one. For each, copy from the explorer:
its size (**bytes**), virtual size (**vB, virtual byte**), weight (**WU, weight unit**) and fee (**satoshi**).

Hand in: a small table of the eight copied numbers with both txids; by hand, for both: check that weight ≈ 4 × virtual size (and explain in one sentence why the explorer rounds the virtual size up), and compute the fee rate = fee ÷ virtual size in sat/vB; and one sentence: why is the virtual size of the SegWit transaction smaller than its real size, while for the legacy one they are equal — and why does that make SegWit cheaper for the same number of real bytes? If your block has no such pair, say so and take the nearest block above H2 that has one.

2.6 — The fee market at three hours of the day [3]

Open the front page of mempool.space three times on **the same day**: in the morning, in the afternoon and late in the evening. Each time take a screenshot with the **clock visible** (the operating-system bar is fine). The page shows the fee estimates for “low”, “medium” and “high” priority (the explorer's words for how fast you want your payment in a block) in sat/vB, and the mempool with the number of blocks of transactions waiting in it.

Time	Low sat/vB	Medium sat/vB	High sat/vB	Blocks waiting in the mempool
morning				
afternoon				
late evening				

Hand in: the three screenshots, the table, and two sentences: why do the numbers move during the day? If all three tiers show the same number, say so, and explain what that tells you about demand for space in blocks.

Part 3 — Mining by hand [30 points]

From Lecture 7. You become a miner for half an hour. No script, no calculator for finding nonces: **by hand**, one try at a time — this is exactly what a mining machine does, only a trillion times faster. Open `https://andersbrownworth.com/blockchain/block` (the demo from the lecture).

3.1 — The Human Miner, one zero [5]

Set **Block = 1**, **Data = D** (your surname in capitals, **exactly as registered in eClass**, example **YUSUPOV**), **Nonce = 1**. Look at the hash. (The page colours the block red until the hash starts with four zeros — ignore the colour; read the first characters of the hash yourself.)
If the hash does not start with `0`, try nonce 2, then 3, and so on.

Hand in: the winning nonce, the number of attempts, and a screenshot of the block with your D, the winning nonce and the hash all visible.

3.2 — The Human Miner, two zeros [8]

Same block, same D. Now continue to count from where you stopped until the hash starts with two zeros, 00. Because you started at nonce 1, your number of attempts is simply your winning nonce.

Do not change D, not even one letter: one changed letter changes the whole hash, and every attempt you made so far is wasted.

Hand in: the winning nonce, the number of attempts, a screenshot; and ✍ by hand: the expected number of attempts for one zero (16) and for two zeros (256) — with the reason (the first character of the hash is one of 16 hex digits, so it is 0 about once in 16 tries; the first two are 00 about once in 16 × 16 = 256 tries — on average, not a promise) — and one sentence: were you lucky or unlucky?

3.3 — Hard to find, easy to check [3]

Take a classmate's (swap results with a classmate) screenshot of 3.2 (or your own, if you have no classmate; then use D = INHA and a nonce you found yourself). Type the classmate's D and nonce once into the page. One attempt checks a result that took hundreds of attempts to find.

Hand in: two sentences only: why is this asymmetry — hard to find, easy to check — what makes proof of work usable?

3.4 ✍ — Read the coinbase of H2 [8]

Open the **first** transaction of block H2: the coinbase transaction
(Lecture 7: the transaction where the miner creates the new coins and has no real input).

Hand in: a screenshot; and, by hand: the halving count (H2 ÷ 210,000, whole part), the **subsidy** at your height (50 BTC halved that many times; for blocks after April 2024 it is 3.125 BTC = 312,500,000 satoshi — check it), the total paid to the miner (the sum of all outputs of the coinbase), the fees = total − subsidy, compared with the total fees shown on the block page; and the name of the mining pool if the explorer shows it.

3.5 — Confirmations grow [6]

Return to the Etherscan page of your transaction 1.1 (to the course address). Look at the field **Block confirmations** at two moments **at least 30 minutes apart**.



Write down the time of each look.

Hand in: two screenshots of the confirmation count, with the clock visible; and by hand: the new blocks (second count − first count), and the seconds per Sepolia block = (minutes between the looks × 60) ÷ new blocks. Example: 150 new blocks in 30 minutes → 1,800 ÷ 150 = 12 seconds. Compare it with Bitcoin's ten minutes in one sentence.
(For Sepolia, say “blocks built on top of yours”.)

Submission checklist

- The ZIP is named `SOC4190_HW2_<StudentID>_<Surname>.zip`.
- `report.pdf` is inside it, and opens.
- Your numbers — H2, D, the txid of T,
  your variant and your Account 1 address — are written at the top of the report.
- Every screenshot shows your own address, H2, D or student ID, as the screenshot rules on page 1 describe.
- No screenshot shows a Secret Recovery Phrase or a private key.
- Every ✍ exercise is a photograph of handwriting, with your name and ID on the sheet.
- `src/` contains `txid.py`, and it runs.
- If you used an AI tool, the first line of the report says which one and for what.